

Análisis y Búsqueda de Respuestas en Datos - Guía Educativa

1. Conceptos Básicos de Computación y Complejidad

1.1. Lógica Algorítmica

- **Definición:** Un algoritmo es un conjunto de instrucciones **precisas, finitas y definidas** que resuelven un problema.
- **Características esenciales:**
 - **Preciso:** Cada paso debe ser inequívoco.
 - **Finito:** Número limitado de pasos.
 - **Definido:** Mismo resultado para mismas entradas.
- **Formas de representación:**
 - **Diagramas de flujo:** Un único nodo de inicio y al menos uno de fin.
 - **Pseudocódigo:** Lenguaje intermedio entre natural y de programación.

Ejemplo práctico (Pseudocódigo):

```
establecer resultado a 0
para i := 1 hasta 100 hacer
    establecer resultado a resultado + i
fin
```

1.2. Combinatoria y Explosión Combinacional

- **Permutaciones:** Ordenar N elementos $\rightarrow N!$ formas.

- Ejemplo: 3 elementos = $3! = 6$ ordenaciones.
- **Variaciones:** Tomar N elementos de M y ordenarlos $\rightarrow V(M,N) = M!/(M-N)!$.
- Ejemplo: Medallas oro/plata entre 4 personas = 12 formas.
- **Combinaciones:** Tomar N elementos de M sin orden $\rightarrow C(M,N) = M!/(N!(M-N)!)$.
- Ejemplo: Elegir 2 países de 4 = 6 formas.
- **Explosión combinatoria:** Crecimiento rápido del espacio de soluciones.
- Ejemplo: Factorial (N!) crece exponencialmente:
 - $N=10 \rightarrow 3.6$ millones
 - $N=100 \rightarrow \sim 9.33 \times 10^{157}$

1.3. Complejidad Computacional

- **Tipos de complejidad:**
- **Tiempo:** Duración de ejecución.
- **Memoria:** Espacio requerido.
- **Clasificación por tiempo:**
- **Polinomial (tratable):** $T(n) = O(n^k) \rightarrow$ Ej: $3n^3+n+8$.
- **Exponencial (intratable):** $T(n) = O(k^n) \rightarrow$ Ej: 2^n .
- **Comparación crítica:** Para n suficientemente grande, polinomial siempre supera a exponencial.

Tabla comparativa: | n | n^5 | 2^n | |---|----|----| | 5 | 3,125 | 32 | | 20 | 3.2×10^6 | 1.05×10^6 | | 40 | 1.02×10^8 | 1.10×10^{12} |

2. Analítica y Minería de Datos

2.1. Diferenciación Conceptual

- **Análisis de Datos:** Identificar hechos, patrones y tendencias para apoyar decisiones.
- **Analítica de Datos:** Proceso más amplio que incluye recolección, procesamiento y aplicación de insights.

2.2. Niveles de Analítica de Datos

1. **Descriptivo:** ¿Qué ha ocurrido? - Ejemplo: Reportes de ventas mensuales.
2. **Diagnóstico:** ¿Por qué ocurrió? - Ejemplo: Análisis de causas de caída en ventas.
3. **Predictivo:** ¿Qué va a ocurrir? - Ejemplo: Modelos de predicción de demanda.
4. **Prescriptivo:** ¿Qué hacer para que algo ocurra? - Ejemplo: Recomendaciones personalizadas de tratamiento médico.

2.3. Metodologías en Minería de Datos

- **SEMMA (SAS):**
 - Sample (Muestreo)
 - Explore (Exploración)
 - Modify (Modificación)
 - Model (Modelado)
 - Assess (Evaluación)
- **CRISP-DM (Estándar industrial):**
 - Comprensión del negocio
 - Comprensión de datos
 - Preparación de datos
 - Modelado
 - Evaluación
 - Despliegue

3. Aplicación en el Sector Sanitario

Transformación mediante datos:

- **Diagnóstico temprano:** Predicción de enfermedades antes de manifestaciones clínicas.
- **Desarrollo de fármacos:** Identificación de moléculas candidatas mediante análisis computacional.

- **Medicina personalizada:** Adaptación de tratamientos según perfil genético y histórico del paciente.

Caso práctico ilustrativo:

El problema de las **Torres de Hanói** (64 discos) demuestra complejidad exponencial ($2^n - 1$ movimientos $\approx 1.84 \times 10^{19}$). Similarmente, en salud: - **Espacio de soluciones** en diseño de fármacos puede ser combinatoriamente enorme. - **Algoritmos eficientes** (polinomiales) son cruciales para análisis de genomas o imágenes médicas.

4. Herramientas y Lenguajes Especializados

R:

- Lenguaje interpretado especializado en estadística y análisis.
- Fortaleza en visualización y paquetes estadísticos.

Python:

- Lenguaje de propósito general con librerías especializadas (Pandas, NumPy, Scikit-learn).
- Versatilidad para integración en pipelines de datos complejos.

Conclusión y Puntos Clave

1. **Fundamentos algorítmicos** son esenciales para comprender límites computacionales en análisis de datos.
2. La **explosión combinatoria** explica por qué algunos problemas en salud (como diseño de fármacos) requieren aproximaciones inteligentes.
3. **Complejidad polinomial vs exponencial** determina la viabilidad práctica de soluciones.
4. La **analítica de datos** en salud avanza desde lo descriptivo hacia lo prescriptivo, permitiendo medicina predictiva y personalizada.

5. Metodologías como **CRISP-DM** proporcionan marques estructurados para proyectos de minería de datos.
6. **Iteración y comprensión del negocio** (como en el caso de Sandra y Fernando) son cruciales para el éxito en ciencia de datos.

Aplicación práctica: En el contexto sanitario, estos principios permiten desde predecir brotes epidemiológicos hasta optimizar recursos hospitalarios, demostrando que el valor de los datos no está en su volumen, sino en nuestra capacidad para extraer insights accionables de manera eficiente.

Resumen Detallado: Analítica de Datos, Niveles, Metodologías y R

1. Analítica de Datos vs. Análisis de Datos

La **Analítica de Datos** es un concepto amplio que engloba todo el ciclo de vida del dato, mientras que el **Análisis de Datos** es una actividad específica dentro de ella.

Actividades clave de la Analítica de Datos:

- **Obtención/Recolección:** Captura de datos desde múltiples fuentes.
- **Limpieza:** Depuración y corrección de datos.
- **Integración:** Unificación de datos heterogéneos.
- **Gobierno de Datos:** Gestión de:
 - Disponibilidad: Acceso en el momento necesario.
 - Usabilidad: Idoneidad para el propósito.
 - Integridad: Exactitud y consistencia.
 - Seguridad: Protección contra accesos no autorizados.
- **Análisis de Datos:** Procesamiento para extraer insights.

Usos comunes:

- **Negocios:** Reducción de costes y soporte a decisiones estratégicas.
- **Científico:** Comprensión de fenómenos para mejorar abordajes.
- **Servicios:** Optimización de costes y calidad.

Aclaración: La limpieza de datos y el Gobierno de Datos son actividades de la **Analítica de Datos**, no del Análisis de Datos. La Analítica de Datos **no** está contenida dentro del Análisis de Datos; es al revés.

2. Niveles de Analítica de Datos

Existen cuatro niveles progresivos en complejidad y valor:

2.1. Análisis Descriptivo - ¿Qué ha ocurrido?

- **Objetivo:** Describir eventos pasados.
- **Resultados:** Reportes estáticos o cuadros de mando.
- **Fuentes:** Sistemas operacionales (CRM, ERP - tipo OLTP).
- **Ejemplos:**
 - Beneficio mensual de los últimos 12 meses.
 - Tendencia de quejas en soporte telefónico.
 - Línea de producto más rentable.

2.2. Análisis Diagnóstico - ¿Por qué ha ocurrido?

- **Objetivo:** Identificar causas de un fenómeno.
- **Enfoque:** Relacionar información de diversas fuentes.
- **Tecnología:** Almacenamiento en estructuras OLAP y herramientas de visualización interactiva.
- **Ejemplos:**
 - Mayor número de quejas desde el Norte vs. Sur de España.
 - Caída en ventas de neveras.
 - Récord de bajas de clientes en un mes.

2.3. Análisis Predictivo - ¿Qué ocurrirá?

- **Objetivo:** Predecir eventos futuros.
- **Técnica:** Modelos predictivos mediante Machine Learning.
- **Aplicación:** Identificación de riesgos y oportunidades.
- **Ejemplos:**
 - Probabilidad de que un cliente responda a una oferta.
 - Eficacia de un tratamiento médico en un rango de edad.
 - Probabilidad de interés en un producto complementario.

2.4. Análisis Prescriptivo - ¿Qué hacer para que ocurra?

- **Objetivo:** Recomendar la mejor acción a tomar.
- **Base:** Se apoya en los resultados del análisis predictivo.
- **Proceso:** Prueba automática de acciones alternativas.
- **Ejemplos:**
 - Producto más adecuado para el mercado canadiense.
 - Mejor mes para lanzar un nuevo servicio.

Relación: Cada nivel es más complejo que el anterior y aporta un mayor valor, construyendo sobre los resultados del nivel precedente.

3. Metodologías en Minería de Datos

La Minería de Datos es una rama de la IA enfocada en obtener valor mediante modelos predictivos, utilizada en los niveles Predictivo y Prescriptivo.

3.1. SEMMA (Sample, Explore, Modify, Model, Assess)

- **Origen:** Lista de pasos secuenciales desarrollada por **SAS Institute**.
- **Fases:** 1. **Sample:** Selección de datos mediante muestreo. 2. **Explore:** Visualización para entender relaciones y anomalías. 3. **Modify:** Selección y transformación de variables. 4. **Model:** Aplicación de técnicas de minería. 5. **Assess:** Evaluación de la utilidad práctica de los modelos.

- **Crítica:** No contempla explícitamente el conocimiento del negocio en la fase de muestreo.
- **Nota:** Sus creadores aclaran que es la organización lógica de su herramienta SAS Enterprise Miner, no una metodología formal.

3.2. CRISP-DM (Cross-industry standard process for data mining)

- **Origen:** Proyecto de la Unión Europea (ESPRIT), considerado estándar.
 - **Característica clave:** Es un proceso **cíclico e iterativo**.
 - **Fases:** 1. Comprensión del negocio. 2. Comprensión de los datos. 3. Preparación de los datos. 4. Modelado. 5. Evaluación. 6. Despliegue.
 - **Ventaja:** Permite retroalimentación entre fases, aplicando lecciones aprendidas en cada iteración.
-

4. R para Analizar Datos - Primeros Pasos

R es un lenguaje de programación interpretado, de alto nivel y código abierto, diseñado específicamente para el análisis de datos.

4.1. Características Principales

- **Interpretado:** No requiere compilación.
- **De alto nivel:** Abstrae detalles de bajo nivel (gestión de memoria).
- **Multiparadigma:** Imperativo, soporta orientación a objetos y programación funcional.
- **Entornos:** Se puede usar el intérprete base, programas en texto plano o IDEs como **RStudio**. También hay intérpretes online (myCompiler, rdrv.io).

4.2. Conceptos Básicos y Ejemplos

- **Hola Mundo y Vectores:** `r sum(1:3) # Crea un vector 1,2,3 y lo suma.`
Resultado: `[1] 6` El operador `:` crea secuencias. `sum()` opera sobre vectores.
- **Ayuda:** `r ?sum # Ayuda sobre la función 'sum' ??plotting # Búsqueda de temas relacionados`

- **Asignación de Variables:** `r x <- 3 # Se prefiere '<-' y = 4 x + y #`
Resultado: `[1] 7`
- **Mostrar Resultados:** `r (x <- 3 + 4) # Los paréntesis fuerzan la`
visualización. Resultado: `[1] 7`
- **Operaciones con Vectores:** `r 1:3 + 4:6 # Suma elemento a elemento.`
Resultado: `[1] 5 7 9` `c(2, 3, 4) - 1 # Resta 1 a cada elemento. Resultado: [1] 1`
`2 3` `c(2, 5, 2, 7) == 3 # Comparación. Resultado: [1] FALSE FALSE FALSE FALSE` La
función `c()` concatena valores en un vector.
- **Propiedades de Vectores:** `r length(1:8) # Longitud del vector. Resultado:`
`[1] 8` `a <- c("blanco", "negro", "azul") length(a) # Número de elementos: [1] 3`
`nchar(a) # Longitud de cada cadena: [1] 6 5 4`
- **Nombres en Vectores:** `r a <- c(manzana=4, naranja=3, mandarina=5) names(a) #`
Muestra los nombres asignados
- **Acceso a Elementos (Indexación):**
- **Índices positivos** (R empieza en 1, no en 0): `r x <- 2:7 x[c(1, 3, 5)] # Accede`
a posiciones 1, 3 y 5. Resultado: `[1] 2 4 6`
- **Índices negativos** (excluye elementos): `r x[c(-1, -3, -5)] # Excluye`
posiciones 1, 3, 5. Resultado: `[1] 3 5 7`
- **Indexación booleana:** `r x[ c(TRUE, FALSE, TRUE, TRUE, FALSE, TRUE) ] #`
Resultado: `[1] 1 3 4 6`
- **Números Especiales:** `r c(-Inf + 1, Inf - 1, Inf - Inf, NA * 5) # Resultado:`
`[1] -Inf Inf NaN NA`
- `Inf` , `-Inf` : Infinito positivo y negativo.
- `NaN` (Not-a-Number): Resultado de una operación inválida.
- `NA` : Valor ausente (Missing Value).

4.3. Aclaraciones Importantes sobre R

- La operación `c(2, 3, 4) - 1` **SÍ** resta 1 a cada elemento del vector.
- El operador `<=` significa "menor o igual que". Para asignación se usa `<-` o `=`.
- R **SÍ** soporta los números especiales `Inf` , `-Inf` , `NaN` y `NA` .

Propósito General: Este documento proporciona una guía estructurada para comprender el ámbito de la Analítica de Datos, sus niveles de profundidad (Descriptivo, Diagnóstico, Predictivo, Prescriptivo), las metodologías clave para implementarla (SEMMA y CRISP-DM), y una introducción práctica al lenguaje R como herramienta fundamental para el análisis. Su objetivo es facilitar el estudio y la aplicación de estos conceptos en contextos reales, desde la toma de decisiones empresariales hasta la investigación científica.

Resumen: Estructuras de Datos y Análisis en R y Python

3.2.- Arrays y Matrices en R

Arrays

- **Definición:** Estructuras de datos multidimensionales y rectangulares (todas las dimensiones deben tener la misma longitud).
- **Creación:** Se usa la función `array()`, especificando:
 - Un vector con los valores (ej: `1:24`).
 - La dimensión con `dim = c(filas, columnas, profundidad, ...)`.
 - Nombres opcionales con `dimnames = list(nombres_filas, nombres_columnas, ...)`.
- **Ejemplo:**

```
r array_3d <- array(1:24, dim = c(2, 3, 4), dimnames = list(c("izquierda", "derecha"), c("primero", "segundo", "tercero"), c("uno", "dos", "tres", "cuatro")))
```
- **Acceso:** Con corchetes `[]`, separando dimensiones por comas. Un espacio vacío selecciona toda la dimensión.
- Ej: `array_3d[, 1:2, c(2, 3)]` selecciona todas las filas, columnas 1 y 2, de las capas 2 y 3.
- **Información:** `dim(array)` devuelve las dimensiones; `length(dim(array))` el número de dimensiones.

Matrices

- **Definición:** Caso especial de arrays limitado a **2 dimensiones**.
 - **Creación:** Función `matrix()` , con argumentos:
 - Vector de datos.
 - Número de filas (`nrow`) o columnas (`ncol`).
 - Nombres con `dimnames` .
 - **Ejemplo:**

```
r matriz <- matrix(1:6, nrow = 3, dimnames = list(c("uno", "dos", "tres"), c("arriba", "abajo")))
```
 - **Operaciones:** Se pueden realizar operaciones de álgebra de matrices (suma, resta, etc.) entre matrices de dimensiones compatibles.
 - **Autoevaluación clave:** Las matrices son bidimensionales, mientras que los arrays pueden tener cualquier número de dimensiones.
-

3.3.- Listas y Dataframes en R

Listas

- **Definición:** Estructuras **unidimensionales** que pueden contener elementos de **diferentes tipos** (vectores, números, matrices, funciones, etc.).
- **Creación:** Función `list()` .

```
r lista <- list(c(1,2,3), 33, matrix(c(6,7,14,1), nrow=2), mean)
```
- **Nombres:** Se pueden asignar con

```
names(lista) <- c("nombre1", "nombre2", ...)
```

 .
- **Acceso:** Similar a los vectores, por índice o por nombre (con `$` si tienen nombre).

Dataframes

- **Definición:** Estructuras **bidimensionales** (como tablas) que pueden almacenar **diferentes tipos de datos por columna**, pero todos los elementos de una misma columna deben ser del mismo tipo. Análogos a una hoja de cálculo.
- **Creación:** Función `data.frame()` , proporcionando vectores (columnas) de igual longitud.

```
r mi_dataframe <- data.frame(x = letters[1:5], y = 1:5, z = 1:5 > 2)
```

- **Acceso:** Como una matriz: `mi_dataframe[filas, columnas]` .
 - **Combinación:**
 - `cbind()` : Une dataframes por **columnas**.
 - `rbind()` : Une dataframes por **filas** (deben tener las mismas columnas).
 - **Autoevaluación clave:** Las listas son unidimensionales y los dataframes bidimensionales.
-

3.4.- Análisis y Visualización de Datos en R

Carga y Exploración de Datos

- **Datasets incluidos:** R tiene muchos conjuntos de datos en paquetes. Se cargan con `data("nombre_dataset", package = "nombre_paquete")` .
- Ej: `data("kidney", package = "survival")` .
- **Primera vista:** `head(dataframe)` muestra las primeras filas.
- **Dimensión:** `dim(dataframe)` devuelve (número de filas, número de columnas).

Estadísticas Básicas

- Se aplican funciones directamente a vectores (que pueden ser columnas de un dataframe).
- **Acceso a columnas:** Usando el operador `$` (ej: `kidney$time`).
- **Funciones comunes:**
- `mean()` : Media.
- `median()` : Mediana.
- `var()` : Varianza.
- `sd()` : Desviación estándar.
- **Resumen rápido:** `summary(dataframe)` proporciona estadísticas descriptivas para todas las columnas (min, max, media, cuartiles, etc.).

Visualización Básica

- **Gráficos dentro de un dataframe:** Se usa `with(dataframe, funcion_grafico(...))` .

- **Tipos de gráficos:**
 - `plot(x, y)` : Diagrama de dispersión.
 - `hist(x)` : Histograma.
 - **Librerías avanzadas:** Para gráficos más complejos y estéticos existen `lattice` y `ggplot2` .
-

4.- Python para Analizar Datos

Contexto y Elección

- Python es un lenguaje **interpretado, de alto nivel y de propósito general**.
- **Ventajas para análisis de datos:**
- Sencillo y muy extendido.
- Gran ecosistema de librerías especializadas.
- **Librerías fundamentales para análisis:** 1. **NumPy:** Proporciona soporte para arrays y matrices multidimensionales, con operaciones numéricas eficientes. 2. **Pandas:** Construida sobre NumPy, ofrece estructuras de datos flexibles (`Series` , `DataFrame`) y herramientas para manipulación y análisis. 3. **Matplotlib:** Librería base para la creación de visualizaciones estáticas.

Primeros Pasos en Python

- **Ejecución:** Se puede usar el intérprete interactivo (IPython), scripts o notebooks (Jupyter).
 - **"Hola Mundo":** `print("¡Hola Mundo!")` .
 - **Ayuda:** La función `help()` abre una utilidad interactiva. `help(funcion)` da información específica.
-

Propósito General y Puntos Clave

Este documento es una guía educativa que introduce las estructuras de datos fundamentales en **R** (vectores, arrays, matrices, listas y dataframes) y su aplicación en el análisis y visualización básica de datos. Simultáneamente, presenta a **Python**

como una alternativa poderosa para el mismo fin, destacando su filosofía y las librerías clave (NumPy, Pandas, Matplotlib) que equiparan sus capacidades a las de R.

Objetivos de aprendizaje: 1. Diferenciar entre arrays (multidimensionales) y matrices (2D) en R. 2. Comprender la flexibilidad de las listas (elementos heterogéneos) versus la estructura tabular de los dataframes. 3. Aplicar funciones básicas de estadística descriptiva y visualización en R. 4. Reconocer a Python, junto con su ecosistema de librerías, como una herramienta válida y popular para el análisis de datos.

Guía de Conceptos Básicos de Python, NumPy y Pandas

1. Fundamentos de Python

1.1. Funciones Esenciales

- `help()` : Proporciona documentación sobre módulos, funciones, clases, etc.
- Ejemplo: `help("modules spam")` muestra información sobre el módulo "spam"
- `print()` : Muestra valores por pantalla
- Ejemplo: `print(3)` → 3

1.2. Asignación de Variables

- Se usa el operador `=` para asignar valores
- Python es de **tipado dinámico**: las variables adquieren su tipo según el valor asignado
- Ejemplo: `x = 3` (entero), `y = "hola"` (cadena), `z = 55` (entero)
- IPython no muestra automáticamente resultados de asignaciones; se debe usar `print()`

1.3. Comentarios

- Se usan con el carácter `#`
- Ejemplo: `x = 3 #Esto es un comentario`

1.4. Operaciones Aritméticas

- Operadores básicos: `+`, `-`, `*`, `/`, `**` (exponenciación)
- Ejemplo: `y ** x` calcula y elevado a x

1.5. Listas

- Secuencias ordenadas que pueden contener diferentes tipos de datos
- **Acceso a elementos:**
- `lista[2]` : elemento en posición 2 (índice comienza en 0)
- `lista[-2]` : segundo elemento desde el final
- `lista[2:4]` : sublista desde posición 2 hasta 3 (4 no incluido)
- **Anidamiento:** Las listas pueden contener otras listas, permitiendo estructuras multidimensionales
- Ejemplo: `lista = [33, [1, 2, 3], ["azul", [77, 88, 99], "verde"]]`

2. Arrays con NumPy

2.1. Introducción a NumPy

- **Problema con listas:** Son ineficientes para análisis de datos debido a que permiten tipos diferentes
- **Solución:** NumPy proporciona el tipo `array` con elementos del mismo tipo
- **Ventaja:** Alto rendimiento mediante operaciones vectorizadas

2.2. Importación

```
import numpy as np # np es el alias convencional
```

2.3. Creación de Arrays

- **Desde listas:** `np.array([1, 2.1, 3, 4.2, 5])`
- Nota: Si hay decimales, todos los elementos se convierten a decimal
- **Funciones especiales:**
- `np.zeros(5, dtype=float)` : Array de 5 ceros como flotantes
- `np.ones((2, 2, 2), dtype=int)` : Array 2x2x2 de unos como enteros
- `np.full((2, 4), 2.78)` : Array 2x4 lleno del valor 2.78
- `np.random.random((2, 2))` : Array 2x2 con valores aleatorios

2.4. Acceso a Arrays

- **Sintaxis:** `x[inicio:final:paso]`
- Valores por defecto: inicio=0, final=tamaño de dimensión, paso=1
- **Ejemplos con array unidimensional:**
- `x[3]` : Elemento en posición 3
- `x[:4]` : Primeros 4 elementos
- `x[4:]` : Elementos desde índice 4
- `x[3:6]` : Elementos entre índices 3 y 5
- `x[1::2]` : Elementos de 2 en 2 desde índice 1
- **Arrays multidimensionales:** Usar `,` para separar dimensiones
- Ejemplo: `x[2:, :2]` (filas desde índice 2, columnas hasta índice 1)

2.5. Vistas vs Copias

- **Vista:** Al obtener una porción (`slice`), se crea una referencia al array original
- Modificar la vista afecta al original
- **Copia:** Usar `.copy()` para crear una copia independiente
- Ejemplo: `z = x[:2, :2].copy()`

2.6. Operaciones Matemáticas

- **Operaciones elemento a elemento:** `x1 + 3` , `x1 * 3` , `x1 / 2`
- **Operaciones entre arrays:** `x1 + x2` , `x1 / x2`

- **Funciones matemáticas:**

- `np.exp2(x1)` : 2 elevado a cada elemento
- `np.power(3, x1)` : 3 elevado a cada elemento
- `np.log2(x1)` , `np.log10(x1)` : Logaritmos

- **Estadísticas:**

- `np.sum(x1)` : Suma total
- `np.min(x1)` , `np.max(x1)` : Mínimo y máximo
- `np.mean(x1)` : Media
- `np.std(x1)` : Desviación estándar
- `np.var(x1)` : Varianza

3. Análisis con Pandas

3.1. Introducción a Pandas

- Librería construida sobre NumPy para manipulación y análisis de datos
- **Estructuras principales:**
- `DataFrame` : Equivalente a tabla bidimensional con etiquetas
- `Series` : Estructura unidimensional (no cubierta en detalle aquí)

3.2. Importación

```
import pandas as pd # pd es el alias convencional
```

3.3. Creación de DataFrames

- Desde listas: `pd.DataFrame(lista, columns=['Fruta', 'Numero'])`
- Ejemplo: `python lista = [['Manzana', 7], ['Pera', 9], ['Mandarina', 11]] df = pd.DataFrame(lista, columns=['Fruta', 'Numero'])`

3.4. Selección de Columnas

- Usar el nombre de la columna entre corchetes

- Ejemplo: `df['Fruta']` selecciona la columna "Fruta"

4. Diferencias Clave con R

- **Índices:** Python comienza en 0, R comienza en 1
- **Rangos:** En Python `inicio:fin` excluye `fin`, en R lo incluye
- **Acceso multidimensional:** En NumPy `x[0,0]`, no `x[0][0]`

5. Recursos Adicionales

- Documentación oficial de NumPy
- Wikipedia: NumPy
- Documentación oficial de Pandas

Nota importante: Esta guía integra conceptos básicos de Python con herramientas especializadas para análisis de datos (NumPy y Pandas), mostrando la progresión desde estructuras nativas hasta librerías optimizadas para computación científica y manipulación de datos.

Resumen: Manipulación de DataFrames en Pandas y Visualización con Matplotlib

1. Manipulación de DataFrames en Pandas

Selección, adición y eliminación de columnas

- **Seleccionar una columna:** Se accede mediante su nombre. `python df['Fruta']`
- **Añadir una columna:** Se asigna una lista a un nuevo nombre de columna. `python df['NuevaCol'] = [3, 5, 7]`
- **Eliminar una columna:** Se usa la palabra clave `del`. `python del df['ColumnaAEliminar']`

Selección, adición y eliminación de filas

- **Tomar filas:** Se usa indexación por rangos (`inicio:fin:paso`), similar a NumPy. `python df[1:3] # Fila 1 a la 2 (sin incluir la 3)`
- **Añadir filas:** Se emplea el método `.append()` para unir DataFrames. `python df = df.append(df2)`
- **Eliminar filas:** Se usa `.drop()` con el índice de la fila a eliminar. `python df = df.drop(1) # Elimina la fila con índice 1` **Nota importante:** Si los índices se repiten (como tras un `append`), `drop` eliminará **todas** las filas con ese índice. Para evitarlo, se pueden definir índices únicos personalizados.

Descripción de datos

- `.describe()` : Proporciona estadísticas descriptivas (count, mean, std, min, percentiles, max) para columnas numéricas. `python df.describe()`

2. Visualización con Matplotlib

Importación y conceptos básicos

- Se importa comúnmente como: `python import matplotlib.pyplot as plt`
- Es una librería versátil que funciona con listas de Python, arrays de NumPy, y DataFrames/Series de Pandas.

Creación de gráficos

1. **Gráfico de líneas (plot):** `python plt.plot(x, y)`
2. **Gráfico de puntos:** Usando el parámetro 'o' . `python plt.plot(x, y, 'o', color="black")`
3. **Otros tipos de gráficos: - Dispersión (scatter):** `plt.scatter(x, y)` - **Barras (bar):** `plt.bar(categorías, valores)`

Gráficos múltiples (subplots)

- Se pueden crear varios gráficos en una misma figura usando `plt.subplot()` .
`python plt.subplot(131) # 1 fila, 3 columnas, posición 1 plt.plot(...)`
`plt.subplot(132) # Posición 2 plt.scatter(...) plt.suptitle('Título general') #`
Título para toda la figura

3. Puntos clave para el estudio

- **Pandas:** Los DataFrames son estructuras flexibles. Se pueden modificar dinámicamente (añadir/eliminar columnas y filas) y explorar rápidamente con `.describe()` .
- **Matplotlib:** Es la librería estándar para visualización. Su función `pyplot` (alias `plt`) ofrece una interfaz sencilla para crear una amplia variedad de gráficos, desde simples líneas hasta figuras complejas con múltiples subgráficos.
- **Interoperabilidad:** Ambas librerías trabajan bien juntas; Matplotlib puede visualizar datos extraídos directamente de DataFrames de Pandas.

Propósito general: Este texto enseña las operaciones fundamentales para manipular y visualizar datos en Python, sentando las bases para el análisis exploratorio de datos (EDA).

